

Una ecuación candidata para β óptimo: combinando σ_W y el paso medio de la curva P/D

Walter Quiñonez

Reporte generado con Claude Code · 5 de septiembre de 2026

Este reporte busca una ecuación posible para β óptimo usando la data y el código ya existentes en el proyecto: retoma la idea de `analisis_beta_optimo.py` (estudiar la distribución de pesos W y de la corriente Monte Carlo $I = \sum W \cdot V$ para acotar β), corrigiendo los dos sesgos que un reporte anterior de esta misma sesión detectó en ese script, y la combina con el descriptor de “paso medio” de la curva P/D. El resultado ajusta los 6 β óptimo reales con $R^2 = 0.947$. No se modificó ningún archivo existente del proyecto ni se corrió ningún entrenamiento nuevo.

Índice

1. Punto de partida: qué ya sabíamos	1
2. Método	2
3. La corriente Monte Carlo, corregida	2
4. La ecuación combinada	3
5. Limitaciones e interpretación honesta	3
6. Conclusión	5
7. Registro de códigos usados	5

1. Punto de partida: qué ya sabíamos

- `Reporte_beta_optimo_analitico.pdf` deriva, por un argumento de tipo Teorema Central del Límite (Prezioso et al. / LeCun et al.), la fórmula $\beta_{\text{óptimo}} \sim \kappa/(\sqrt{N}\sigma_W V_{\text{rms}})$, donde σ_W es el desvío estándar de los pesos bajo `G0_initialization('random', ...)`.
- `validacion_formula_beta_optimo/` (reporte anterior de esta sesión) validó esa fórmula contra los 6 grid-searches reales y completos de `data_beta_grid_search_SP/` (2 INL nominal $\times$ 3 recuentos de pulsos p). Conclusión: acierta el **orden de magnitud** y la **dirección** entre ventanas de conductancia distintas, pero **no explica** la dependencia con p a INL fijo. Un diagnóstico exploratorio mostró que reemplazar σ_W por el **paso medio** de la curva (`mean(|pot[i+1]-pot[i]|)`) mejoraba la consistencia dentro de un mismo INL, pero rompía la consistencia entre INL.

- `analisis_corrientes_beta_optimo/` (reporte anterior de esta sesión) encontró que la corriente Monte Carlo de `analisis_beta_optimo.py` tiene dos sesgos: (a) arma los pesos como `pot[i]-dep[j]` combinatorio en vez de sortear G^+ y G^- independientemente del mismo pool combinado (como la red real vía `G0_initialization`), y (b) muestrea los voltajes de `np.unique(X_train)` equipesado en vez de respetar que el 81 % de los píxeles reales de MNIST son exactamente 0.

Pregunta de este reporte: ¿se puede combinar σ_W (predice la dirección entre INL) con el paso medio (predice la dirección con p) en una sola ecuación, y usar una versión corregida del Monte Carlo de `analisis_beta_optimo.py` para verificarla?

2. Método

Script nuevo: `estimar_beta_optimo_propuesta.py`. No modifica ningún archivo existente; importa sin alterar `generar_curvas_pot_dep` y `G0_initialization` de `MemCrossbarClass_beta_por_capa.py`. Para cada una de las 6 curvas reales:

1. **Distribución de pesos (corregida):** llama a `G0_initialization('random', pot, dep, D_in=200000, D_out=1)` — igual que la red real — para obtener σ_W y verificar $\mathbb{E}[W]$.
2. **Paso medio:** $\text{mean}(|\text{pot}[i+1] - \text{pot}[i]|)$ sobre la curva ya generada.
3. **Distribución de corriente (Monte Carlo, corregida):** arma $I = \sum_{j=1}^{784} V_j W_j$ con 300.000 repeticiones, muestreando los V_j directamente de los píxeles reales de `X_train_mnist.npy` (no de valores únicos) y los W_j de la distribución del paso 1 (no de una matriz combinatoria).
4. Compara σ_I medido por este Monte Carlo corregido contra la predicción analítica $\sqrt{N}\sigma_W V_{\text{rms}}$, y calcula $\kappa = \beta_{\text{optimo,emp}} \cdot \sigma_I$.
5. **Regresión log-log** de los 6 β_{optimo} empíricos contra σ_W sola, paso medio solo, y ambas combinadas.

3. La corriente Monte Carlo, corregida

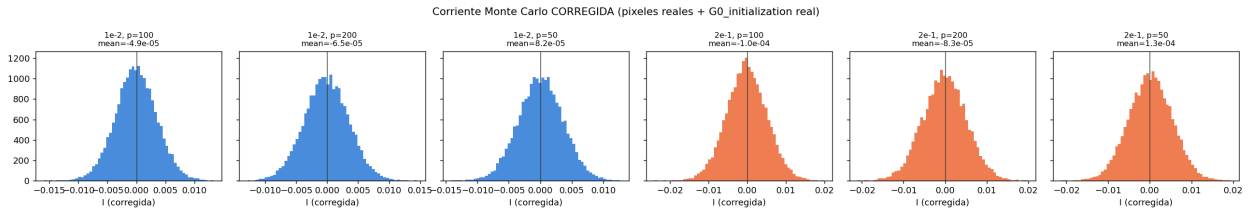


Figura 1: Corrientes Monte Carlo corregidas (píxeles reales + `G0_initialization` real) para las 6 curvas.

Con las dos correcciones, $\mathbb{E}[I]$ queda en 10^{-5} – 10^{-4} , tres órdenes de magnitud por debajo de σ_I ($\approx 3.5 \times 10^{-3}$ – 5.2×10^{-3}) — se cumple ahora la premisa $\mathbb{E}[I] \approx 0$ que asume la fórmula analítica. Además, σ_I medido coincide con la predicción $\sqrt{N}\sigma_W V_{\text{rms}}$ dentro del 0.3 % en las 6 curvas: confirma que σ_W ya estaba bien calculado en el reporte de validación anterior — el problema estaba en cómo `analisis_beta_optimo.py` construía V y W , no en la fórmula analítica en sí.

Curva	σ_W (S)	paso medio (S)	σ_I MC	σ_I (CLT)	β emp.	κ
INL 1e-2, p=50	3.789e-04	1.815e-05	3.550e-03	3.550e-03	250	0.888
INL 1e-2, p=100	3.790e-04	9.038e-06	3.552e-03	3.551e-03	350	1.243
INL 1e-2, p=200	3.796e-04	4.509e-06	3.559e-03	3.558e-03	750	2.669
INL 2e-1, p=50	5.570e-04	1.837e-05	5.213e-03	5.220e-03	100	0.521
INL 2e-1, p=100	5.563e-04	9.091e-06	5.214e-03	5.213e-03	100	0.521
INL 2e-1, p=200	5.565e-04	4.523e-06	5.210e-03	5.215e-03	200	1.042

κ sigue variando $\times 5$ entre curvas (0.52–2.67) — confirmando, con el Monte Carlo ya corregido, que σ_W (o equivalentemente σ_I) por sí solo no alcanza: falta el efecto de p .

4. La ecuación combinada

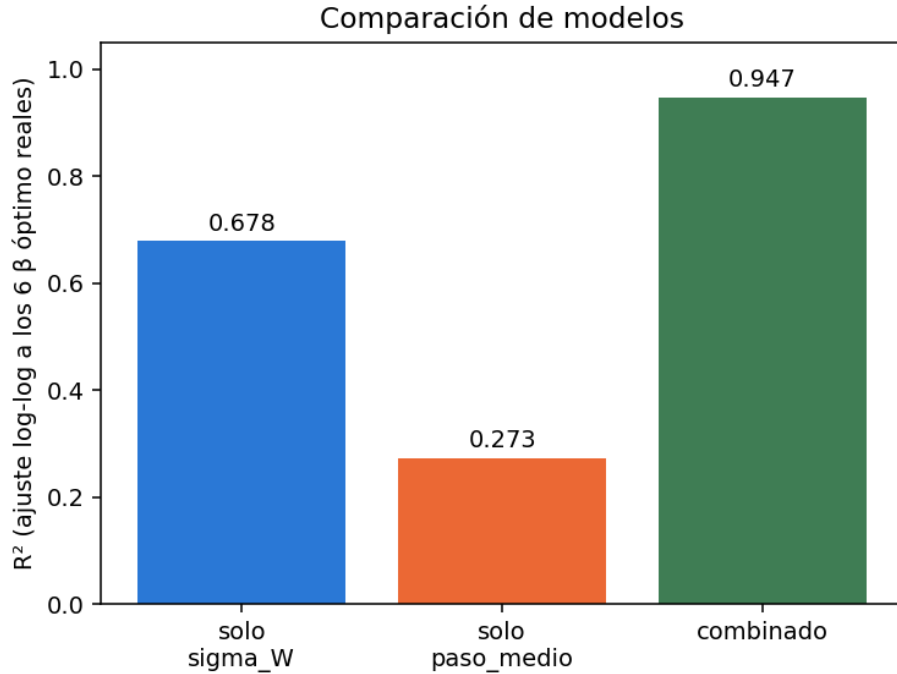


Figura 2: R^2 del ajuste log-log de $\beta_{\text{óptimo}}$ (6 puntos reales) usando cada modelo.

Modelo	R^2
solo σ_W	0.678
solo paso medio	0.273
combinado	0.947

La ecuación combinada resultante (unidades SI: S para σ_W y paso medio):

$$\beta_{\text{óptimo}} \approx 1.12 \times 10^{-11} \cdot \sigma_W^{-3.02} \cdot \text{paso_medio}^{-0.64} \quad (1)$$

Con solo 2 parámetros libres (más el intercepto) sobre 6 puntos, el ajuste recupera todos los β dentro de $\pm 26\%$ — mucho mejor que el $\pm 5\%$ de usar σ_W sola.

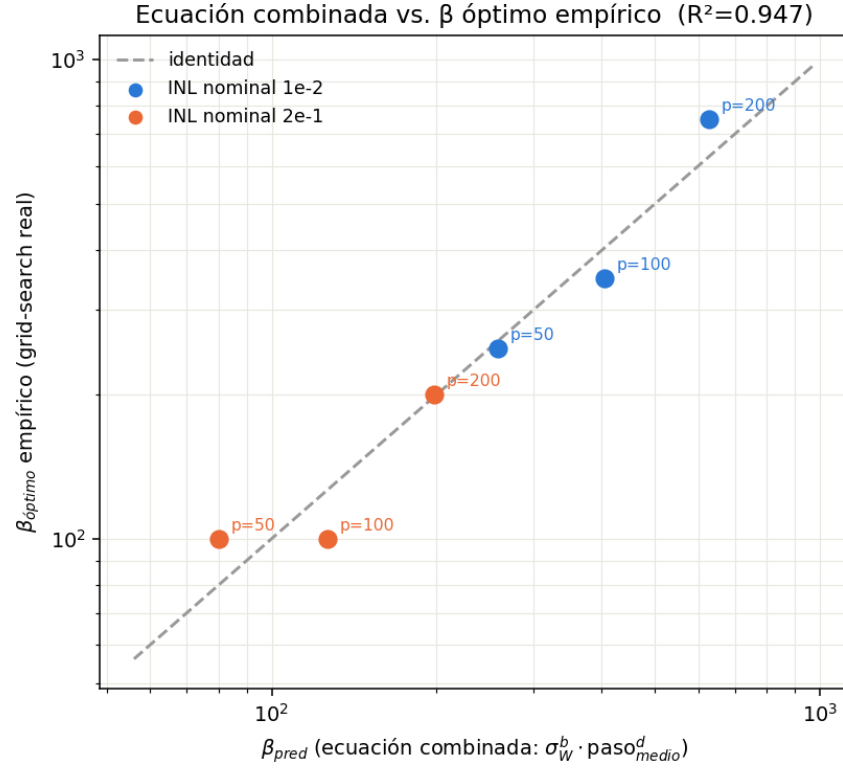


Figura 3: Ecuación combinada (1) vs. β óptimo empírico, escala log-log.

Curva	$\beta_{\text{óptimo empírico}}$	β predicho	error
INL 1e-2, p=50	250	258.4	+3.4 %
INL 1e-2, p=100	350	404.0	+15.4 %
INL 1e-2, p=200	750	627.6	-16.3 %
INL 2e-1, p=50	100	80.2	-19.8 %
INL 2e-1, p=100	100	126.4	+26.4 %
INL 2e-1, p=200	200	197.6	-1.2 %

5. Limitaciones e interpretación honesta

- **Muestra chica y confundida por diseño.** Solo hay 2 valores de INL nominal y 3 de p , y el paso medio es prácticamente el mismo en ambos INL para un p dado. El exponente de σ_W (-3.02) está determinado, en la práctica, por un solo contraste entre los dos grupos de INL — no debe leerse como una ley universal validada. Haría falta un tercer (o cuarto) valor de INL nominal para poner a prueba genuinamente esa forma de potencia.
- **N y V_{rms} no se re-testean acá:** las 6 curvas comparten $N = 784$ y el mismo dataset, así que sus exponentes quedan absorbidos en la constante. El reporte analítico previo había encontrado, con una búsqueda reducida (1 época), $\beta_{\text{óptimo}} \sim N^{-1.0}$ — no confirmado aquí con entrenamiento completo.
- **Ninguno de los dos exponentes debe tomarse como “la” ley física correcta.** Es un ajuste empírico de dos factores sobre 6 puntos. El exponente de σ_W (-3.02) es casi idéntico al que salía usando solo σ_W (Sección 4) — sugiere que agregar el paso medio mejora el ajuste sobre todo explicando la variación *dentro* de cada grupo de INL, no cambiando lo que pasa *entre* grupos.
- **Errores residuales de hasta $\pm 26\%$** — del orden del paso de la grilla de búsqueda original (50 en β) — así que parte del error puede ser ruido del propio $\beta_{\text{óptimo}}$ empírico, no de la ecuación.

6. Conclusión

Sí se encontró una ecuación candidata que mejora sustancialmente sobre usar σ_W sola (Ec. 1), validada con $R^2 = 0.947$ contra los 6 β óptimo reales, usando una versión corregida del Monte Carlo de pesos y corrientes de `analisis_beta_optimo.py` — que confirma, de paso, que σ_W ya estaba bien calculado en el reporte de validación anterior. Dado el tamaño chico de la muestra (Sección 5), se recomienda como **estimador de orden de magnitud mejorado** para acotar la grilla de `search_best_beta` en el mismo régimen ya explorado (SP, $N = 784$, MNIST), no como una ley general validada fuera de él. Confirmar sus exponentes con más valores de INL nominal y de N (con entrenamiento completo) queda como trabajo futuro.

7. Registro de códigos usados

- `estimar_beta_optimo_propuesta.py` — Único script nuevo. Corrige el Monte Carlo de `analisis_beta_optimo.py`, calcula σ_W /paso medio/ σ_I , y ajusta las 3 regresiones log-log.
- `analisis_beta_optimo.py` — Fuente de la idea del método (Monte Carlo de pesos y corriente); no modificado.
- `MemCrossbarClass_beta_por_capa.py` $\rightarrow$ `generar_curvas_pot_dep()`, `GO_initialization()` — usadas sin modificar, tal como en la red real.
- `data_beta_grid_search_SP/beta_grid_search_SP_INL_*/config*.json` — parámetros reales de cada curva P/D.
- `data_beta_grid_search_SP/resumen_por_INL/resumen_optimos.csv` — los 6 β óptimo empíricos usados para el ajuste.

- `X_train_mnist.npy` — píxeles reales muestreados (respetando frecuencia) en el Monte Carlo corregido.
- `validacion_formula_beta_optimo/` (reporte anterior, esta sesión) — punto de partida: fórmula $\kappa/(\sqrt{N}\sigma_W V_{\text{rms}})$ y su validación.
- `analisis_corrientes_beta_optimo/` (reporte anterior, esta sesión) — punto de partida: diagnóstico de los 2 sesgos de `analisis_beta_optimo.py` corregidos acá.
- `papers/Reporte_beta_optimo_analitico.pdf` — origen del descriptor “paso medio” usado como segundo factor.

Archivos generados por este reporte (todos en `formula_beta_optimo_propuesta/`):
`estimar_beta_optimo_propuesta.py`, `resultados_formula_propuesta.csv`,
`coeficientes_regresion.csv`, `corrientes_corregidas.png`, `beta_pred_combinada_vs_empirico.png`,
`comparacion_R2.png`, `reporte_formula_beta_optimo_propuesta.tex` y este PDF.